

Project Report

Title Page

Title: MockMate AI: AI-Powered Mock Interview Platform

Submitted By: Rohit Patidar

Roll No.: IC-2K21-70

Institute: IIPS DAVV

Program/Department: MCA

Academic Year: [Academic Year]

Guide/Faculty Mentor: [Name, Designation]

Date of Submission: [Date]

Declaration

I, Rohit Patidar (Roll No. IC-2K21-70), hereby declare that the project titled "MockMate AI: AI-Powered Mock Interview Platform" is my original work and has not been submitted elsewhere for any academic award. The report is prepared as part of the MCA curriculum at IIPS DAVV and all sources of information used in this report are duly acknowledged.

Plagiarism Report

- **Tool Used:** [Plagiarism Tool]
 - **Similarity Index:** [XX%]
 - **Date:** [Date]
-

Acknowledgement

I express my sincere gratitude to my faculty guide [Name] for the guidance and support throughout this project. I also thank the department and IIPS DAVV for providing the resources and environment to complete this work. Lastly, I appreciate my peers and family for their encouragement and motivation.

Table of Contents

1. Abstract

2. Executive Summary

3. Introduction

4. Analysis

5. Project Planning

1

2

3

4

5

6. System Design 6

7. Software Development Methodology 8

8. System Implementation 9

9. System Testing 12

10. Output Forms & Reports 13

11. Limitations 14

12. Conclusion 15

13. Bibliography 16

14. Appendices 17

Abstract

MockMate AI is an AI-powered mock interview platform designed to help students and early-career professionals practice interview skills in a realistic, conversational setting. The system provides a structured interview session where an AI interviewer asks domain-specific questions and follows up based on user responses. The platform supports multiple domains and difficulty levels, offers a live chat interface, and generates an evaluation score at the end of the session. The solution is built using a modern web stack with a React frontend and a Node.js/Express backend, integrating with a large language model for interview interactions and evaluation. The system emphasizes usability, real-time feedback, and progress tracking through a personalized dashboard. MockMate AI aims to reduce interview anxiety by enabling consistent practice, and it provides actionable insights for improvement.

Executive Summary

MockMate AI is a full-stack web application that simulates a mock technical interview. Users select a domain and difficulty level, interact with an AI interviewer through chat and optional voice input, and receive a score at the end of the session. The platform stores interview history and aggregates performance data on the user dashboard. The system is built with React, Tailwind CSS, and Vite on the frontend, and Node.js, Express, MongoDB, and JWT authentication on the backend. The AI workflow uses an interview prompt to ask concise follow-up questions and a separate evaluation endpoint to compute the final score. The project delivers a practical, scalable practice tool with clear opportunities for future enhancement.

1. Introduction

MockMate AI addresses the need for accessible interview practice tools for students and job seekers. Traditional mock interviews are limited by availability, time, and cost. This project provides an AI-driven interview environment that simulates a real interview flow and offers measurable feedback. The system focuses on conversational skill, understanding of concepts, and communication clarity rather than coding tasks.

2. Analysis

Problem Statement

Students often lack consistent access to realistic interview practice. Existing solutions may not provide adaptive questioning or structured performance tracking.

Objectives

- Provide AI-driven interview sessions with domain-specific questions.
- Offer follow-up questions instead of full explanations to mimic real interviews.
- Track and store interview results for users.
- Provide a clean, responsive user interface.

Feasibility

- **Technical:** Implemented with common web technologies and AI APIs.
- **Operational:** Usable by students with minimal setup.
- **Economic:** Low cost of deployment with scalable cloud infrastructure.

3. Project Planning

Scope

- User authentication and dashboard.
- Interview setup and session management.
- Chat-based interaction with AI interviewer.
- Score generation and interview history storage.

Timeline (High-Level)

- Requirements and analysis
- UI design and frontend development
- Backend APIs and database integration
- AI prompt engineering and evaluation flow
- Testing and stabilization

4. System Design

Architecture

- **Frontend:** React + Tailwind CSS + Vite.
- **Backend:** Node.js + Express.
- **Database:** MongoDB (user profiles, interviews).
- **AI Service:** Groq API for interview responses and evaluations.

Key Modules

- **Authentication:** JWT-based login/signup.
- **Interview Engine:** Session management and AI prompts.
- **Evaluation Engine:** Score generation using AI evaluation.
- **Dashboard:** Stats and interview history display.

Data Flow (High Level)

1. User logs in.
2. User starts interview with selected domain and difficulty.

3. Frontend sends user responses to backend.
4. Backend calls AI model for follow-up questions.
5. At completion, backend evaluates and stores results.
6. Dashboard displays updated stats.

5. Software Development Methodology

An incremental Agile approach was followed. The project was split into sprints covering UI design, backend APIs, AI integration, and evaluation logic. Each sprint delivered a working component that was iteratively refined based on testing and feedback.

6. System Implementation

Frontend

- React components for pages: Home, Dashboard, Interview.
- Tailwind CSS for responsive, modern UI.
- Protected routes for authenticated access.

Backend

- Express REST APIs for authentication, interview sessions, evaluation, and result storage.
- MongoDB schema to store user interviews, total interviews, and average scores.

AI Integration

- System prompt designed to ask concise follow-up questions.
- Separate evaluation prompt generates a structured score.

7. System Testing

Testing Types

- **Functional Testing:** Login, interview flow, end-to-end session completion.
- **API Testing:** Interview endpoints, evaluation endpoint, result persistence.
- **UI Testing:** Responsiveness on mobile and desktop.

Results

All critical flows were verified manually. Minor UI adjustments were made to prevent layout overflow during long chat sessions.

8. Output Forms & Reports

- **Login/Signup Forms:** User authentication flow.
- **Interview Room:** Live chat and session status.
- **Dashboard:** Total interviews, average score, interview history table.
- **Score Summary:** Final score after interview completion.

9. Limitations

- AI responses depend on the external API availability.
- Voice recognition accuracy varies by browser/device.
- Current scoring is AI-evaluated and may require manual calibration.

10. Conclusion

MockMate AI provides a practical and scalable mock interview environment that helps users build confidence and track progress. The system successfully integrates AI-driven questioning with user-friendly design and persistent performance tracking. Future improvements can include richer analytics, customizable interview templates, and improved evaluation transparency.

Bibliography

- <https://react.dev/>
 - <https://expressjs.com/>
 - <https://www.mongodb.com/>
 - <https://vitejs.dev/>
 - <https://tailwindcss.com/>
 - <https://groq.com/>
-

Appendices

Appendix A: API Endpoints (Summary)

- `POST /api/auth/signup`
- `POST /api/auth/login`
- `GET /api/auth/me`
- `GET /api/auth/profile/:id`
- `POST /api/interview`
- `POST /api/interview/evaluate`
- `POST /api/interview/complete`
- `POST /api/interview/end`

Appendix B: Database Schema (User)

- name, email, password
- interviews: domain, difficulty, score, duration, date
- totalInterviews, averageScore

Appendix C: Environment Variables

- `MONGODB_URI`
- `JWT_SECRET`
- `GROQ_API_KEY`
- `GROQ_MODEL`